Unit 10: Trees (Week 8)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: Binary Trees

Section 10.1

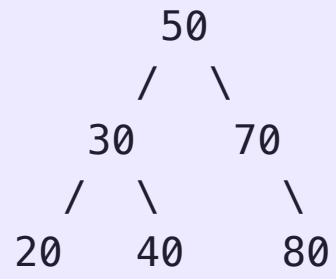
🤔 A linked list node points to one successor. What if each node pointed to two?

You'd have branching structure — but is that useful, and what guarantees does it give you?

What properties make a binary tree different from a general graph?

10.1 Binary Tree Basics

Hierarchical structure; each node has at most two children: a **left child** and a **right child**.

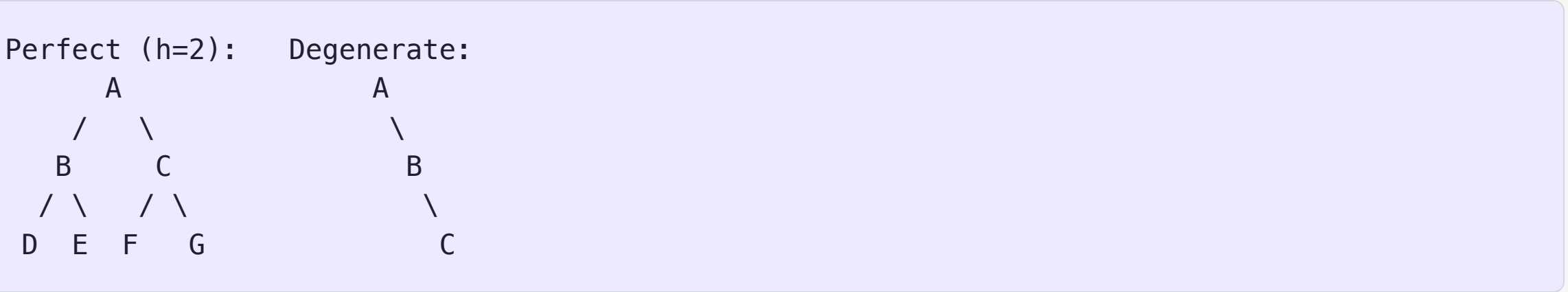


Term	Definition
Root	Topmost node; no parent
Leaf	Node with no children
Height	Longest path from root to a leaf
Level	Distance from root (root = level 0)

10.1 Tree Shape and Height

The shape of a binary tree determines the cost of every operation performed on it.

- **Full:** every node has 0 or 2 children
- **Complete:** all levels filled except possibly the last, filled left-to-right
- **Perfect:** all levels completely filled; exactly $2^{h+1} - 1$ nodes at height h
- **Degenerate:** every node has exactly one child — a linked list in disguise



10.1 Binary Tree Basics

A tree with n nodes has exactly **n–1 edges** — no cycles possible.

> A perfect binary tree of height h has height O(log n). A degenerate tree has height O(n).

Part 2: Applications of Trees

Section 10.2

🤔 Where do binary trees actually appear in software you use every day?

Think about how a compiler processes `3 + 4 * 2`, or how a file system stores folders.

Can you name three domains where a tree structure appears naturally?

10.2 Trees in the Wild

Trees appear wherever there is **hierarchical or recursive structure**.

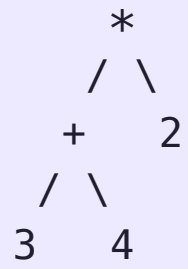
Domain	Tree type	What nodes represent
File systems	General tree	Directories / files
Compilers	AST (binary/n-ary)	Expressions, statements
Databases	B-tree / B+ tree	Index pages
Networking	Spanning tree	Routers
AI game search	Game tree	Board states
Data compression	Huffman tree	Character encodings

The BST you build this unit is the conceptual foundation of `std::set`, `std::map`, and database indices in general.

10.2 Expression Trees

A compiler parses `(3 + 4) * 2` into an **abstract syntax tree** where:

- Internal nodes are **operators**
- Leaf nodes are **operands**



Evaluating the tree post-order (left, right, root) gives the correct result: 3, 4, +→7, 2, *→14. The tree encodes operator precedence structurally — no parentheses needed.

Part 3: Binary Search Trees

Section 10.3

🤔 A sorted array supports O(log n) binary search. But insertion costs O(n).

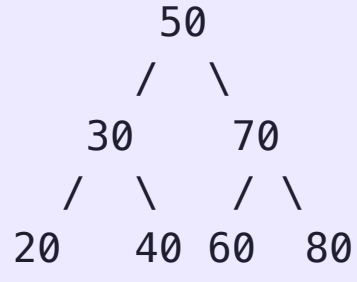
A linked list supports O(1) insertion. But search costs O(n).

BST gets O(log n) for both, without sorting on every insert.

10.3 The BST Property

A **Binary Search Tree** enforces one invariant at every node:

> All values in the **left subtree** < node value < all values in the **right subtree**



This property holds **recursively** for every subtree — not just at the root. It's what makes binary search possible at every level.

Part 4: BST Search

Section 10.4

🤔 You're searching for value 60 in a BST. At each node, what decision do you make?

The BST property gives you a binary choice at every step.

How many nodes do you visit in the worst case on a tree of height h?

10.4 BST Search Algorithm

At each node, compare the target to the current value:

- **Equal** → found
- **Target < current** → go left
- **Target > current** → go right
- **Null** → not found

```
Node* search(Node* node, int key) {
    if (!node || node->val == key) return node;
    if (key < node->val) return search(node->left, key);
    return search(node->right, key);
}
```

Each comparison eliminates an entire subtree. Cost: **O(h)** where h is the tree height.

10.4 Search Trace

Searching for 60 in this BST:

```
       50          Step 1: 60 > 50 → go right
      /  \
    30    70       Step 2: 60 < 70 → go left
   /  \   /
  20  40 60        Step 3: 60 == 60 → found ✅
```

3 comparisons for 7 nodes. For a balanced BST with n nodes, height ≈ $\log_2 n$ — so search visits at most $\lfloor \log_2 n \rfloor + 1$ nodes.

Part 5: BST Insertion

Section 10.5

🤔 Where does a new value belong in a BST?

The BST property must hold after insertion — and the new node must end up somewhere specific.

Can you always insert as a leaf? Why or why not?

10.5 BST Insertion

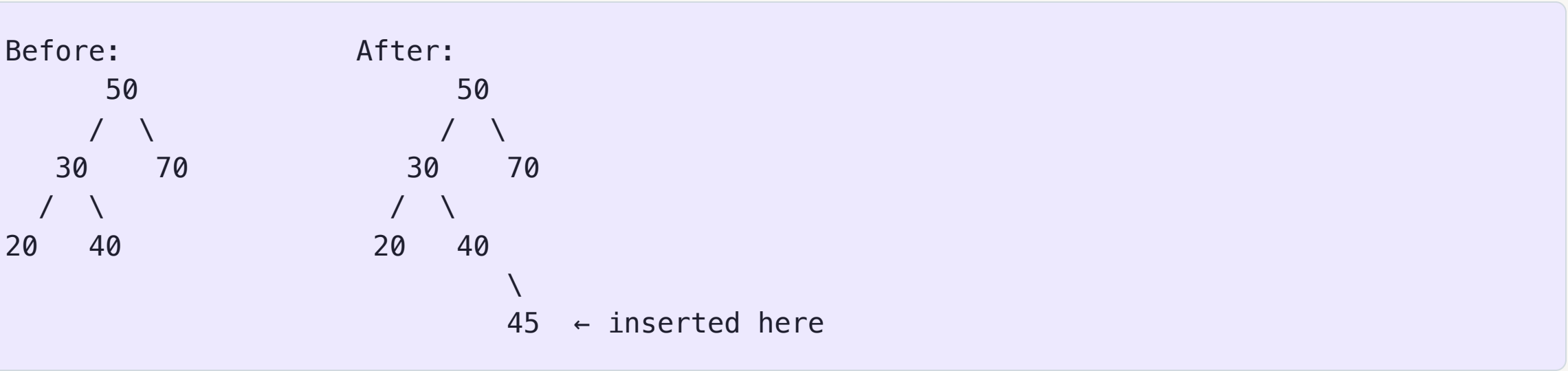
Search for the new key as if it already existed. When you reach a null pointer, insert there.

```
Node* insert(Node* node, int key) {
    if (!node) return new Node(key);
    if (key < node->val) node->left  = insert(node->left,  key);
    else if (key > node->val) node->right = insert(node->right, key);
    return node;
}
```

New nodes are always inserted as **leaves** — the tree's existing structure is never rearranged. Cost: **O(h)**.

10.5 Insertion Trace

Inserting 45 into the BST:



45 > 30 → right. 45 > 40 → right. 40's right child is null → insert.

Part 6: BST Removal

Section 10.6

🤔 Deleting a leaf is easy. But what if you delete a node with two children?

You can't just remove it: the subtrees below it would be disconnected.

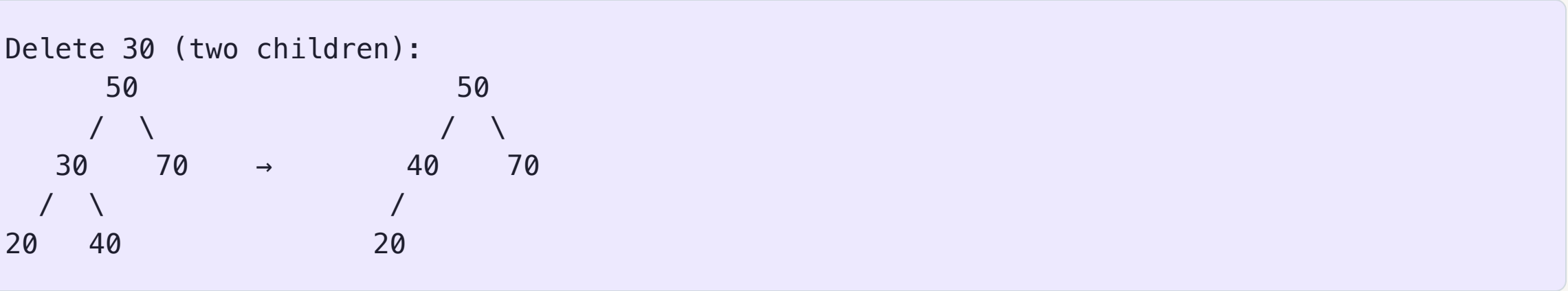
What value could replace the deleted node while preserving the BST property?

10.6 BST Removal: Three Cases

Case 1 — Leaf node: simply remove it.

Case 2 — One child: replace the node with its child.

Case 3 — Two children: replace the node's value with its **in-order successor** (smallest value in the right subtree).



30 is replaced by 40 (in-order successor). 40 is then removed from its original position.

Part 7: BST Traversal

Section 10.7

🤔 There's no single "correct" order to visit a tree's nodes.

Sometimes you need values smallest-to-largest.

Sometimes you need to process children before parents.

Sometimes the opposite.

What are the three natural recursive traversal orders for a binary tree?

10.7 The Three Traversals

Traversal	Order	BST Application
In-order	Left → Node → Right	Sorted ascending
Pre-order	Node → Left → Right	Prefix / copy tree
Post-order	Left → Right → Node	Delete tree / evaluate expression

```
void inOrder(Node* n) {
    if (!n) return;
    inOrder(n->left); cout << n->val << " "; inOrder(n->right);
}
```

10.7 Traversal Trace

```
Tree:          50
              /  \
            30    70
           /  \
         20   40

In-order:   20 30 40 50 70   ← sorted!
Pre-order:  50 30 20 40 70   ← root first
Post-order: 20 40 30 70 50   ← root last
```

In-order traversal of any BST always produces values in **sorted ascending order**, which is a direct consequence of the BST property.

Part 8: Height and Insertion Order

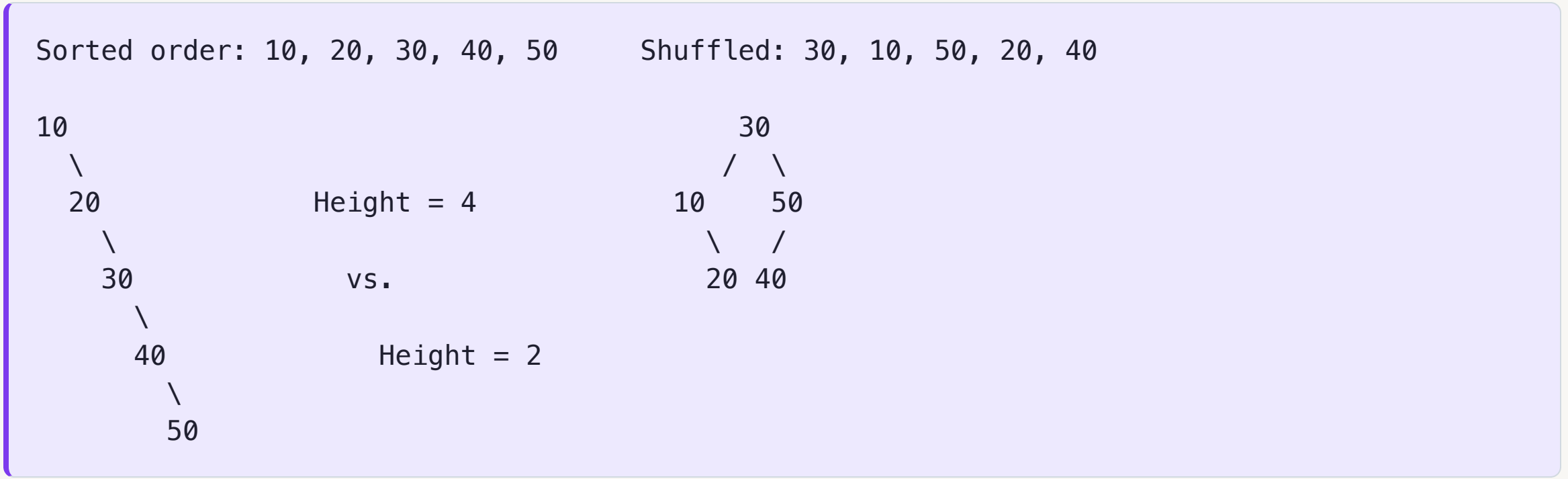
Section 10.8

🤔 Insert 10, 20, 30, 40, 50 into a BST in that order.

Then insert the same values as 30, 10, 50, 20, 40.

Draw both trees. What is the height of each?

10.8 Insertion Order Determines Height



The sorted-order tree is a degenerate linked list: O(n) height, O(n) operations. The shuffled tree is balanced: O(log n) height, O(log n) operations. **Same data, radically different performance.**

10.8 Expected Height from Random Insertions

If n keys are inserted in a **uniformly random order**, the expected height is **O(log n)** — provably. The probability of degenerate behavior is exponentially small.

But in practice, data often arrives in partially sorted order — log files, timestamps, auto-incremented IDs. A plain BST offers no protection.

> Self-balancing BSTs (AVL trees, Red-Black trees) restructure on insert and delete to maintain O(log n) height **regardless** of insertion order — at the cost of more complex code. `std::map` and `std::set` in C++ use Red-Black trees internally.

Part 9: Parent Node Pointers

Section 10.9

🤔 In our BST so far, each node only knows about its children.

Some operations — like finding a node's in-order successor without restarting from the root — require navigating upward.

What is the simplest change to the node structure that enables upward traversal?

10.9 Adding a Parent Pointer

```
struct Node {
    int val;
    Node *left, *right, *parent;
    Node(int v) : val(v), left(nullptr), right(nullptr), parent(nullptr) {}
};
```

Every insert must now set `parent` on the new node. Every rotation or restructuring must update parent pointers: which is why self-balancing trees are more complex.

10.9 In-Order Successor in a BST

What is an In-Order Successor?

The **in-order successor** of a node is:

> The node visited immediately after it in an in-order traversal.

In-order traversal:

```
left → root → right
```

In a BST, this means:

> The next larger value.

Two Cases

To find the successor of node `x`:

1. `x` has a right subtree
2. `x` has no right subtree

Case 1 — Right Subtree Exists

If `x.right` exists:

> The successor is the **leftmost node** in the right subtree.

Example

```
      20
     /  \
   10    30
        /
      25
```

Find successor of `20`

1. Go right → `30`
2. Then go left as far as possible → `25`

Successor:

```
25
```

Why Does This Work?

In-order traversal visits:

```
left subtree
current node
right subtree
```

So the next node after `x` must be:

> the smallest node in the right subtree.

Case 2 — No Right Subtree

If `x.right` does NOT exist:

> Move upward using parent pointers.

Stop when:

> you move up from a LEFT child to its parent.

That parent is the successor.

Example

```
      20
     /  \
   10    30
     \
     15
```

Find successor of `15`

- No right subtree
- Move up to `10`
    - came from RIGHT child → continue
- Move up to `20` (came from LEFT child)

Successor: 20

Intuition

If there is no right subtree:

- You already finished this subtree
- Keep moving upward
- The first ancestor larger than `x` becomes the successor

Full Example

```
          20
        /    \
      10      30
     /  \    /  \
    5   15  25  35
```

In-order traversal:

```
5 → 10 → 15 → 20 → 25 → 30 → 35
```

Example: Successor of 10


```
10 has right subtree
```

Go right → `15`

Leftmost node there = `15`

Successor:

```
15
```

Example: Successor of 15

```
15 has no right subtree
```

Move upward:

```
15 → 10 → 20
```

- `15` was a RIGHT child
- `10` was a LEFT child

Successor:

```
20
```

Part 10: Set ADT

Section 10.10

🤔 A mathematical set has no duplicates and supports membership testing, insertion, and deletion.

You could implement this with a sorted array or a hash table.

What does a BST offer that neither of those does?

10.10 Set ADT

A **set** is an abstract data type supporting:

Operation	Description
`insert(x)`	Add x if not present
`remove(x)`	Remove x if present
`contains(x)`	True if x is in the set
`min()` / `max()`	Smallest / largest element
`successor(x)`	Next larger element

A BST implements all of these in **O(h)** time — and uniquely supports `min`, `max`, and `successor` efficiently, which hash tables cannot.

10.10 BST vs. Hash Table for Sets

Property	BST (balanced)	Hash Table
`insert` / `remove` / `contains`	O(log n)	O(1) average
`min` / `max`	O(log n)	O(n)
`successor` / `predecessor`	O(log n)	O(n)
Sorted iteration	O(n)	O(n log n)
Worst case	O(log n) guaranteed	O(n) (collisions)

> Use a hash table when you need fast membership testing and don't care about order. Use a BST when you need **ordered operations** — range queries, sorted output, nearest neighbor.

Part 11: BST Recursion

Section 10.11

10.11 Recursion in BSTs

BST operations follows this shape:

```
T(h) = T(h−1) + O(1)
```

One constant-time comparison, then recurse into a subtree of height h−1. This unrolls to **T(h) = O(h)**.

For a balanced tree, h = O(log n) → operations cost **O(log n)**.
For a degenerate tree, h = O(n) → operations cost **O(n)**.

The recursive structure also makes **divide-and-conquer** on trees natural: each subtree is an independent subproblem of the same type.

10.11 Computing Tree Properties Recursively

```
int height(Node* n) {
    if (!n) return -1;
    return 1 + max(height(n->left), height(n->right));
}

int size(Node* n) {
    if (!n) return 0;
    return 1 + size(n->left) + size(n->right);
}
```

Part 14: Tries

Section 10.14

🤔 You need to store a dictionary of 100,000 words and support prefix search.

"Find all words starting with 'pre'" should be fast — not a scan of all 100,000 entries.

What tree structure would let you share the common prefix 'pre' across all matching words?

10.14 The Trie

A **trie** (prefix tree) stores strings by their characters. Each path from root to a marked node spells a word. Nodes with the same prefix share the same path.

```
Words: "cat", "car", "card", "care", "bat"

          root
         /    \
        c      b
        |      |
        a      a
       / \     |
      t   r    t*
     *   / \
        d*  e*
```

`*` marks end-of-word. All words starting with "ca" live under the same subtree.

10.14 Trie Node

```
struct TrieNode {
    TrieNode* children[26] = {};
    bool isEnd = false;
};
```

Each node holds up to 26 child pointers (one per letter). `isEnd` marks whether the path to this node forms a complete word.

Most children are `nullptr` for typical text — space use is proportional to the number of **characters stored**, not the alphabet size squared.

10.14 Trie Insert and Search

```
void insert(TrieNode* root, const string& word) {
    TrieNode* cur = root;
    for (char c : word) {
        if (!cur->children[c-'a']) cur->children[c-'a'] = new TrieNode();
        cur = cur->children[c-'a'];
    }
    cur->isEnd = true;
}
```

Return false at the first missing child. Cost for a word of length L: **O(L)**, independent of dictionary size.

10.14 Trie vs. BST for Strings

Property	BST of strings	Trie
Search / Insert	O(L log n)	O(L)
Prefix search	O(n) scan	O(L + matches)
Space	O(n · L)	O(total characters)

Here, n means: the number of strings stored in the data structure.

And L means:the length of the string being searched/inserted.

Tries power autocomplete, spell checkers, and IP routing tables. Any application with heavy **prefix query** workloads should reach for a trie before a BST.

Unit 10 Summary

Concept	Key idea
Binary tree	At most 2 children; height determines all costs
BST property	Left < node < right, recursively
Search / Insert / Delete	O(h) — O(log n) balanced, O(n) degenerate
In-order traversal	Always produces sorted output
Insertion order	Determines height; sorted input → degenerate
Parent pointer	Enables upward traversal; complicates balancing
Set ADT	BST adds ordered ops that hash tables can't match
Trie	Prefix-sharing tree; O(L) ops independent of n

Lab 10 Wednesday! 😄 Have a great day!